Okayish Chat

Project Report — Web Application Development

WAD Course · April 2026

Stack	Python · FastAPI · PostgreSQL · Redis · llama-cpp-python
UI Mode	SPA — Single-Page Application
Architecture	MCS — Model · Controller · Service
Auth	JWT access tokens + Redis-backed opaque refresh tokens (30-day TTL)
LLM	Local GGUF model via llama-cpp-python (CPU inference, ChatML prompt)
Bonus	Typewriter output visualisation · Auto-refresh · Token rotation

1. Architecture Overview

Okayish Chat is a **Single-Page Application** — the frontend is plain HTML/CSS/JS served from FastAPI's StaticFiles mount. Because the UI is a SPA, the backend follows the **MCS (Model–Controller–Service)** pattern: routers are thin, all business logic lives in service modules.

Layer	Location	Responsibility
Model	`app/models/`	SQLAlchemy ORM definitions, Alembic-managed migrations
Controller	`app/controllers/`	FastAPI routers — parse requests, delegate to services, return responses
Service	`app/services/`	All business logic: auth, JWT, Redis sessions, LLM calls, DB queries

JWT + Redis Session Flow

Every protected route passes through **dependencies.py**, which reads the Bearer token, decodes the JWT (HS256, 30-minute expiry), and loads the matching User from the database. Refresh tokens are **opaque random strings**, not JWTs — stored in Redis under `session:<token>` with a 30-day TTL. On each refresh the old token is deleted and a new one issued (**token rotation**).

Step	Action
1	Client sends credentials → POST /auth/login or /auth/register
2	Server returns { access_token (JWT, 30 min), refresh_token (opaque, 30 days) }
3	Client stores both in localStorage; attaches access_token as Authorization: Bearer
4	On 401 response, client calls POST /auth/refresh with the refresh_token
5	Server verifies token in Redis, deletes it, issues a fresh pair (rotation)
6	GitHub OAuth users follow the same path — tokens issued in the /callback redirect

2. API Structure

The API is built with FastAPI and fully documented at `http://localhost:8000/docs`. Routes marked ■ require a valid JWT Bearer token.

Auth · /auth

Method	Endpoint	Description	Auth
POST	/auth/register	Register new user, returns token pair	—
POST	/auth/login	Login with username + password	—
POST	/auth/refresh	Rotate refresh token, return new pair	—
GET	/auth/github	Redirect to GitHub OAuth authorization page	—
GET	/auth/github/callback	Handle GitHub callback, issue tokens to SPA	—
GET	/auth/me	Return current authenticated user profile	■ JWT

Chats · /chats

Method	Endpoint	Description	Auth
GET	/chats	List all chats for the current user	■ JWT
POST	/chats	Create a new chat thread	■ JWT
DELETE	/chats/{chat_id}	Delete chat and all its messages (cascade)	■ JWT

Messages · /chats/{chat_id}/messages

Method	Endpoint	Description	Auth
GET	/chats/{chat_id}/messages	Retrieve full message history for a chat	■ JWT
POST	/chats/{chat_id}/messages	Send message → LLM generates reply → both saved	■ JWT

Pydantic Schemas

UserRegister	username, email (optional), password

UserLogin	username, password
UserOut	id, username, email, created_at
TokenResponse	access_token, refresh_token
RefreshRequest	refresh_token
ChatCreate	title
ChatOut	id, user_id, title, created_at
MessageSend	content
MessageOut	id, chat_id, role, content, created_at

3. Code Structure

The project is organised as a standard Python package under **app/**, split cleanly into the three MCS layers. Static frontend files live in **frontend/** and are served by FastAPI's StaticFiles mount at `/static`.

```
okayish_chat/
  app/
    controllers/
      auth.py
      chat.py
      message.py
    models/
      user.py
      chat.py
      message.py
    schemas/
    services/
      auth_service.py
      chat_service.py
      message_service.py
      llm_service.py
    config.py
    database.py
    dependencies.py
    redis_client.py
    main.py
  frontend/
    index.html
    login.html
    register.html
    chat.html
  alembic/versions/
  .env
  requirements.txt
  model.gguf
```

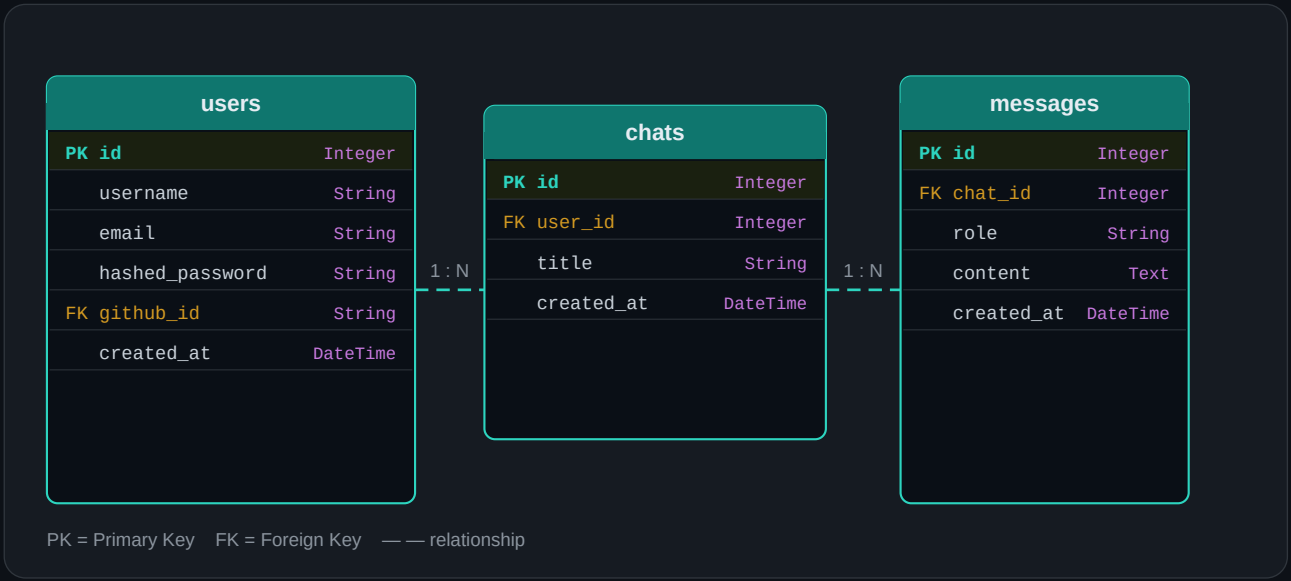
main.py	App entry point — registers all routers, mounts /static, serves index redirect.
controllers/auth.py	Registration, login, token refresh, GitHub OAuth callback, /me endpoint.
controllers/chat.py	List, create and delete chat threads (CRUD).
controllers/message.py	Retrieve message history and send new messages to the LLM.
services/auth_service.py	bcrypt hashing, JWT encode/decode, Redis session creation/rotation, GitHub user upsert.
services/llm_service.py	Lazy-loads GGUF model on first request (singleton). Exposes generate_response() and stream_response().
services/message_service.py	Saves user message, calls LLM, saves assistant reply, returns both.
services/chat_service.py	All DB CRUD operations for the Chat model.
database.py	SQLAlchemy engine + session factory + declarative Base.
redis_client.py	Single Redis connection pool initialised from REDIS_URL env var.
config.py	pydantic-settings class — reads all secrets and config from .env file.
dependencies.py	FastAPI Depends() helper — validates Bearer JWT, loads User from DB.

LLM Integration

llm_service.py uses **llama-cpp-python** to run a GGUF model entirely on CPU. The model loads once on first request and is kept as a module-level singleton. The ChatML prompt template (`<|im_start|>user…<|im_end|>`) is used with stop tokens to prevent runaway generation. The service also exposes **stream_response()**, a generator that yields tokens via llama-cpp-python's `stream=True` mode, ready for future SSE integration.

4. Database Schema

PostgreSQL is the primary database. All schema changes are applied via **Alembic** migrations (initial revision `d6dd5b713066`). The schema consists of three tables connected by two one-to-many relationships.



Entity-Relationship Diagram — users → chats → messages (1:N each)

users

Column	Type	Nullable	Notes
id	Integer	NO	Primary key, auto-increment, indexed
username	String	NO	Unique, indexed — the login identifier
email	String	YES	Unique, indexed — optional for password-based accounts
hashed_password	String	YES	bcrypt hash — NULL for GitHub-only accounts
github_id	String	YES	Unique GitHub user ID — NULL for local accounts
created_at	DateTime	YES	UTC timestamp, defaults to now()

chats

Column	Type	Nullable	Notes
id	Integer	NO	Primary key, auto-increment, indexed
user_id	Integer	NO	FK → users.id, cascade delete

Column	Type	Nullable	Notes
title	String	YES	Conversation title; auto-renamed from first user message
created_at	DateTime	YES	UTC timestamp, defaults to now()

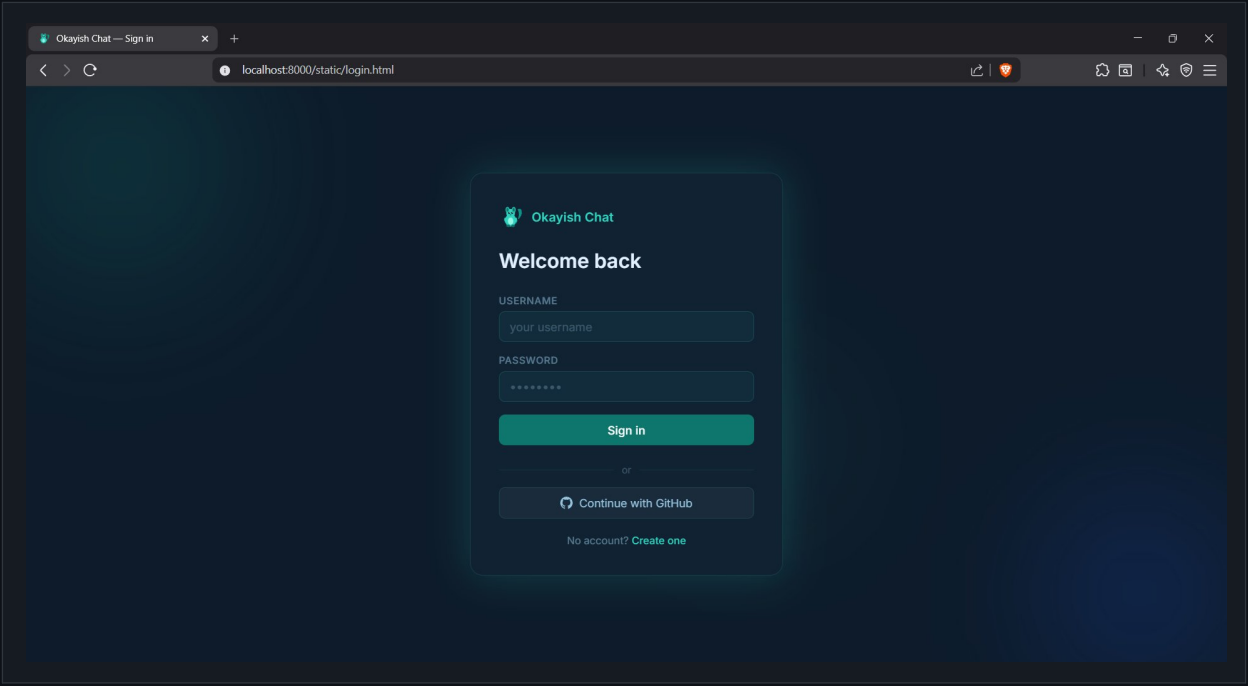
messages

Column	Type	Nullable	Notes
id	Integer	NO	Primary key, auto-increment, indexed
chat_id	Integer	NO	FK → chats.id, cascade delete
role	String	NO	Either 'user' or 'assistant'
content	Text	NO	Full message body (no length limit)
created_at	DateTime	YES	UTC timestamp, defaults to now()

5. User Interface

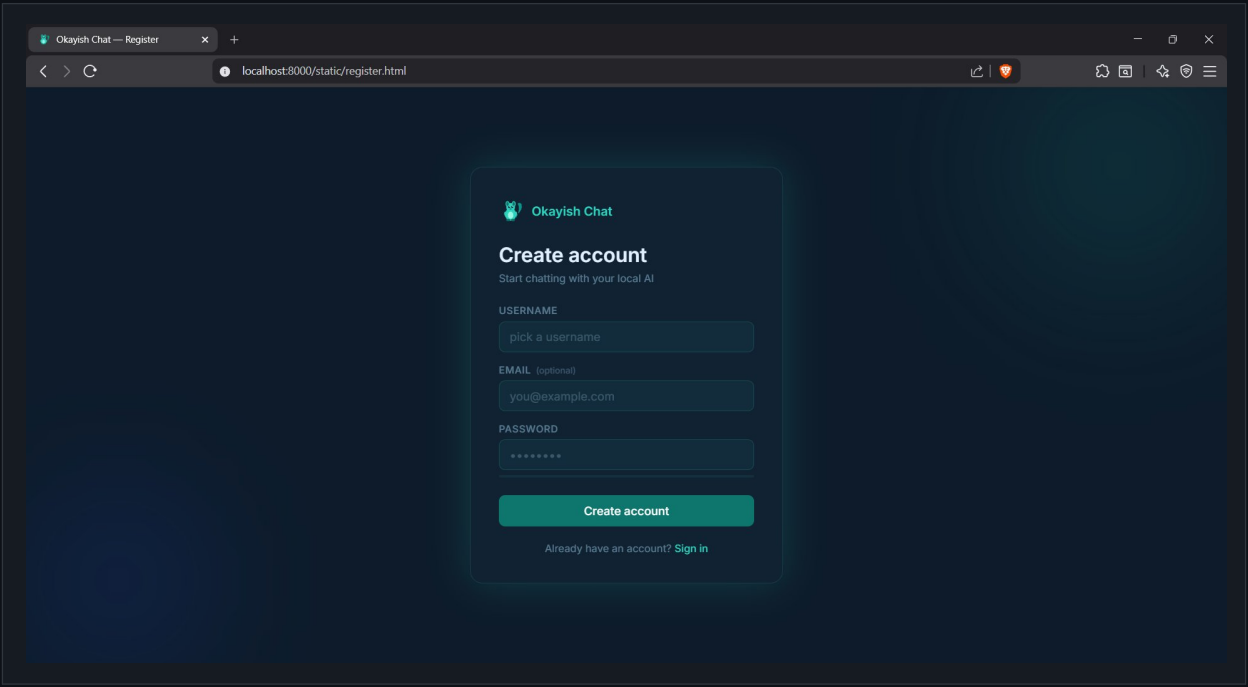
The SPA frontend consists of four pages: **index.html** (auth redirect), **login.html**, **register.html**, and the main **chat.html**. Chat tabs run across the top bar; conversations are selected by clicking a tab. The interface uses a dark teal theme with Inter font.

Sign In



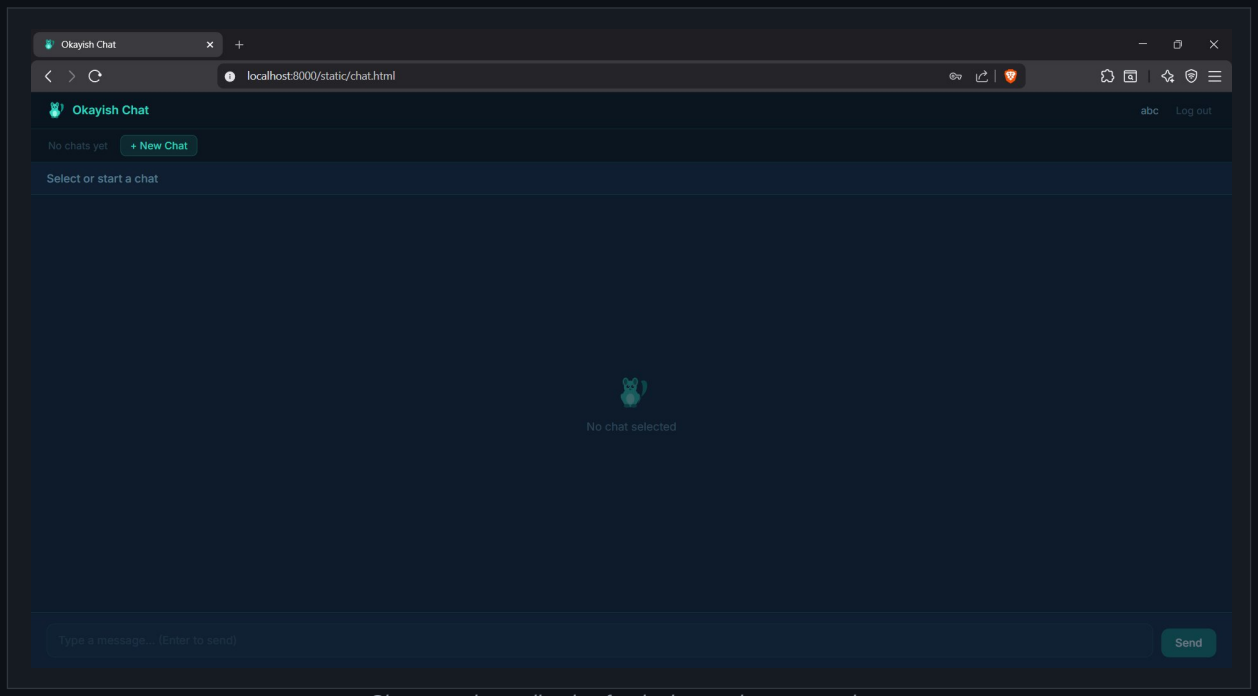
Login page — username/password form with GitHub OAuth option

Register



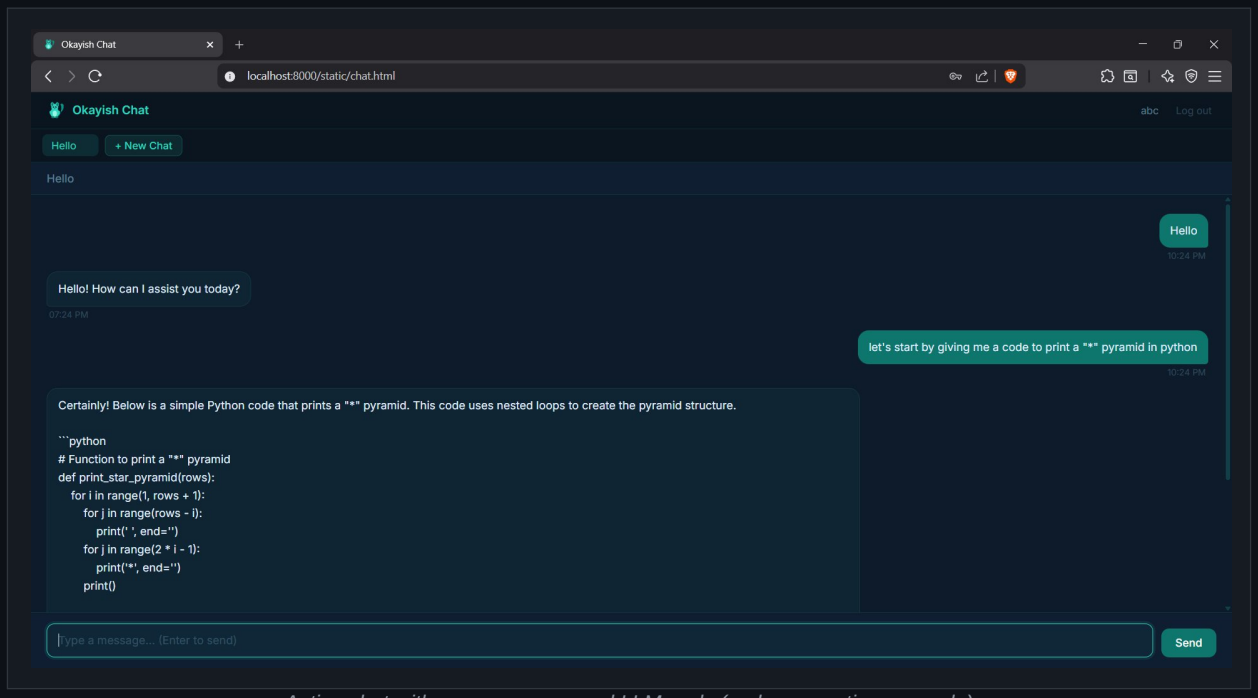
Registration page — username (required), optional email, password

Chat — Empty State



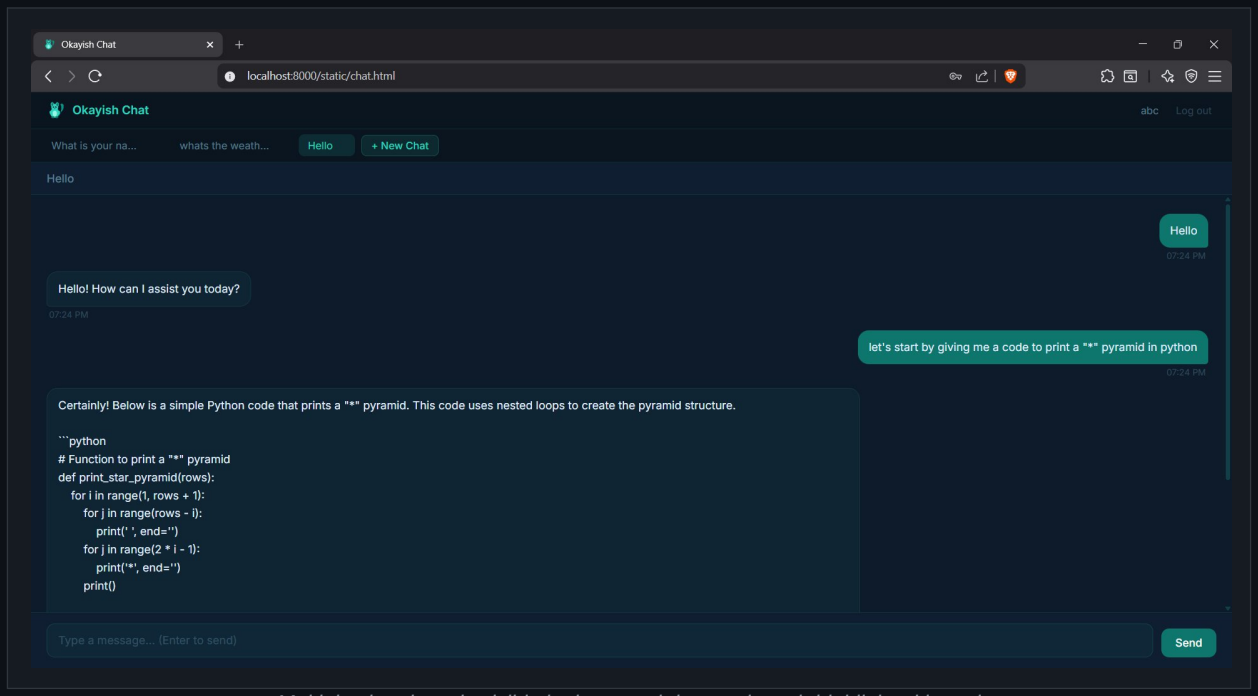
Chat page immediately after login, no chats created yet

Chat — Active Conversation



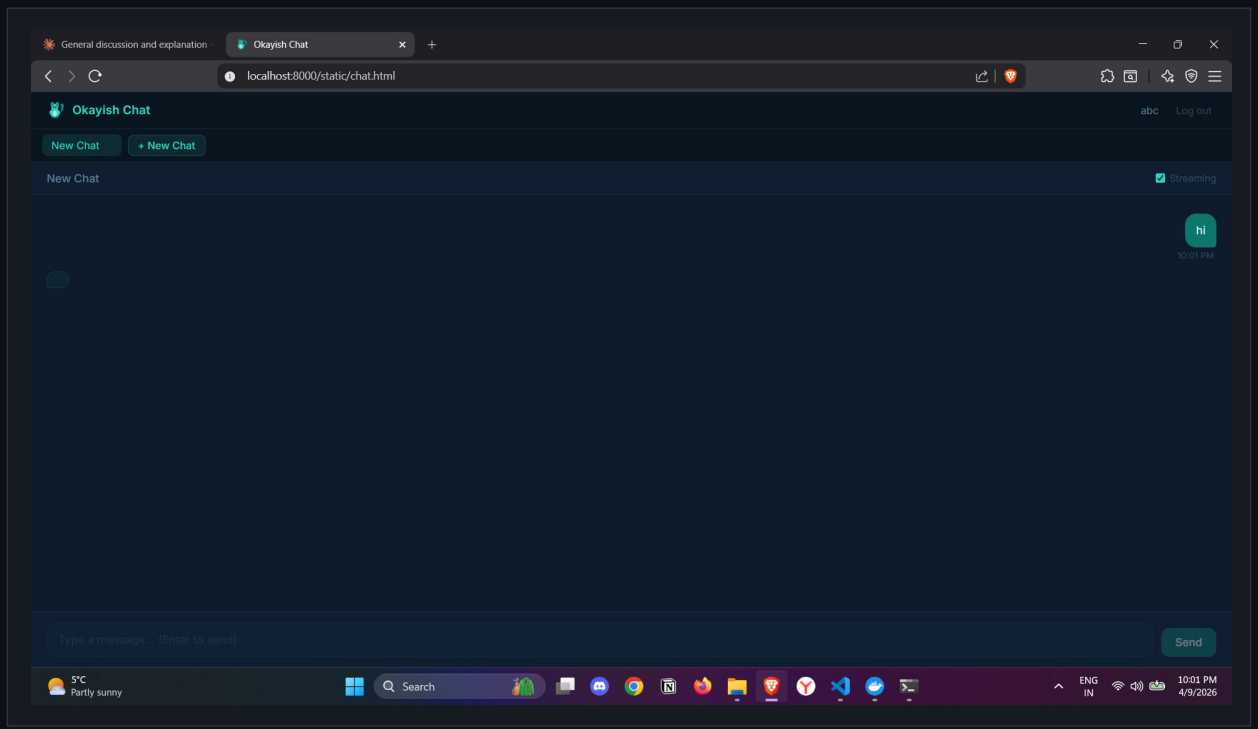
Active chat with user message and LLM reply (code generation example)

Multiple Chat Tabs



Multiple chat threads visible in the top tab bar; active tab highlighted in teal

Chat — In Use



Chat page during an active session showing the message flow

6. API Documentation (Swagger UI)

FastAPI auto-generates interactive OpenAPI docs at `http://localhost:8000/docs`. All endpoints, schemas, and auth requirements are visible and testable there.

Group	Method	Path	Summary
auth	POST	/auth/register	Register
auth	POST	/auth/login	Login
auth	POST	/auth/refresh	Refresh
auth	GET	/auth/github	Github Login
auth	GET	/auth/github/callback	Github Callback
auth	GET	/auth/me	Me 🔒
chats	GET	/chats	List Chats 🔒
chats	POST	/chats	Create Chat 🔒
chats	DELETE	/chats/{chat_id}	Delete Chat 🔒
messages	GET	/chats/{chat_id}/messages	Get Messages 🔒
messages	POST	/chats/{chat_id}/messages	Send Message 🔒
default	GET	/	Root (redirect to /static/index.html)

7. Bonus Features Implemented

All three bonus requirements from the specification are implemented and functional.

★ Streaming output visualisation

The frontend renders each LLM reply with a character-by-character typewriter effect (typewriterReveal() in chat.html). Speed adapts to response length so short replies feel snappy and long ones stay readable. A blinking cursor tracks the write position, matching the spec requirement to 'visualise tokens arriving incrementally in the UI'.

★ Auto-refresh of the chat view

startAutoRefresh() polls the active chat's messages endpoint every 15 seconds while a chat is open. If the message count has changed — for example from another browser session — the view re-renders silently. A small '■ refreshing' indicator appears in the header during the poll. This matches the spec note that 'periodically reloading the page to refresh the chat view is acceptable'.

★ Refresh token rotation

Every call to POST /auth/refresh invalidates the old token in Redis with r.delete() before issuing a new one. This means a stolen refresh token becomes useless the moment the legitimate client uses it next — a security practice that goes beyond the base requirement of simply storing tokens with a TTL.

Soft deadline: 14 May 2026 · Hard deadline: 18 May 2026 · Submitted via GitHub repository